Network layer

Network Layer

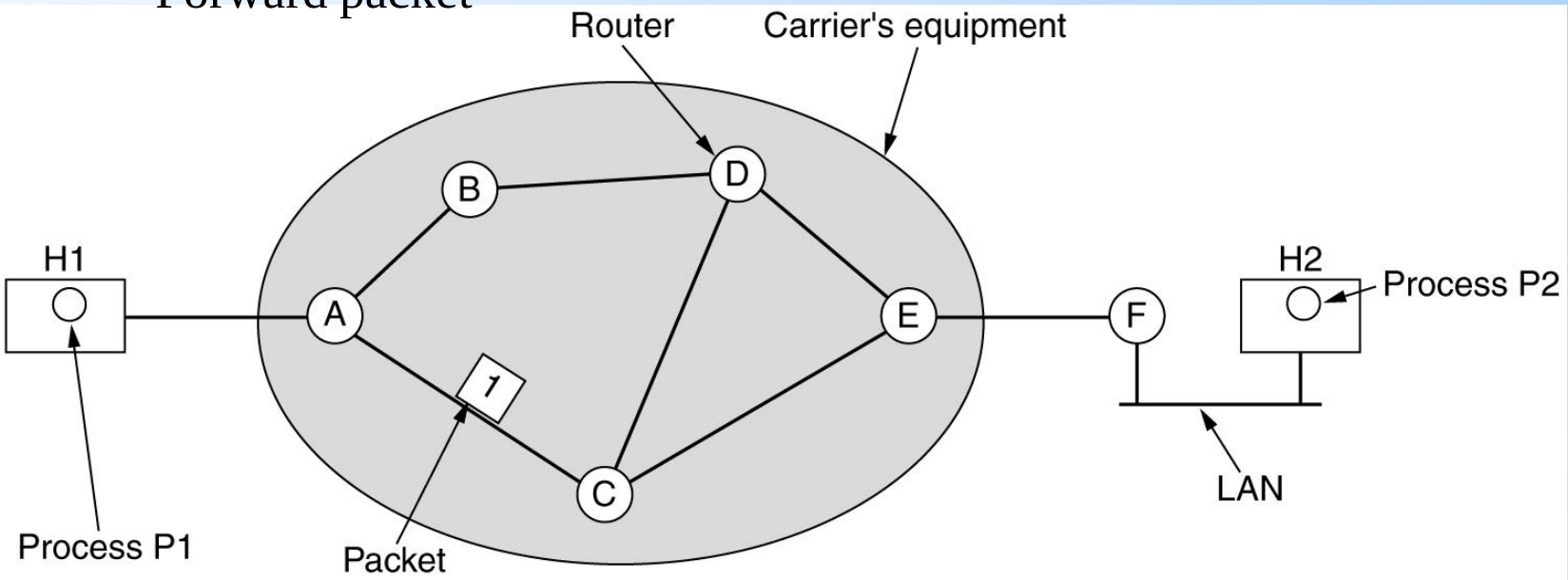
- Design issues
- Routing
- Congestion
- Internetworking
- Internet Protocols
- Multimedia or QoS

Design issues

- Goal of layer: get packets from source host to destination host
  - Routing: should know about topology of subnet
  - Congestion: should avoid overloading some communication lines and routers
  - Quality of service: offer the appropriate service
  - Internetworking: deal with network differences, if source and destination are connected to different networks

Design issues

- Store-and-forward packet switching
  - Equipment of: carrier <> customer
  - Algorithm at router
    - Receive packet
    - Check packet (e.g. checksum)
    - Forward packet



Design issues: services

- Interface
  - Interface between carrier and customer
  - Designed with following goals in mind:
    - Services should be independent of the subnet technology
    - Transport layer should be shielded from the number, type, topology of the subnets
    - Network addresses should use a uniform numbering plan, even across LANs and WANs
- Connections?
  - Connection-oriented <> Connectionless!

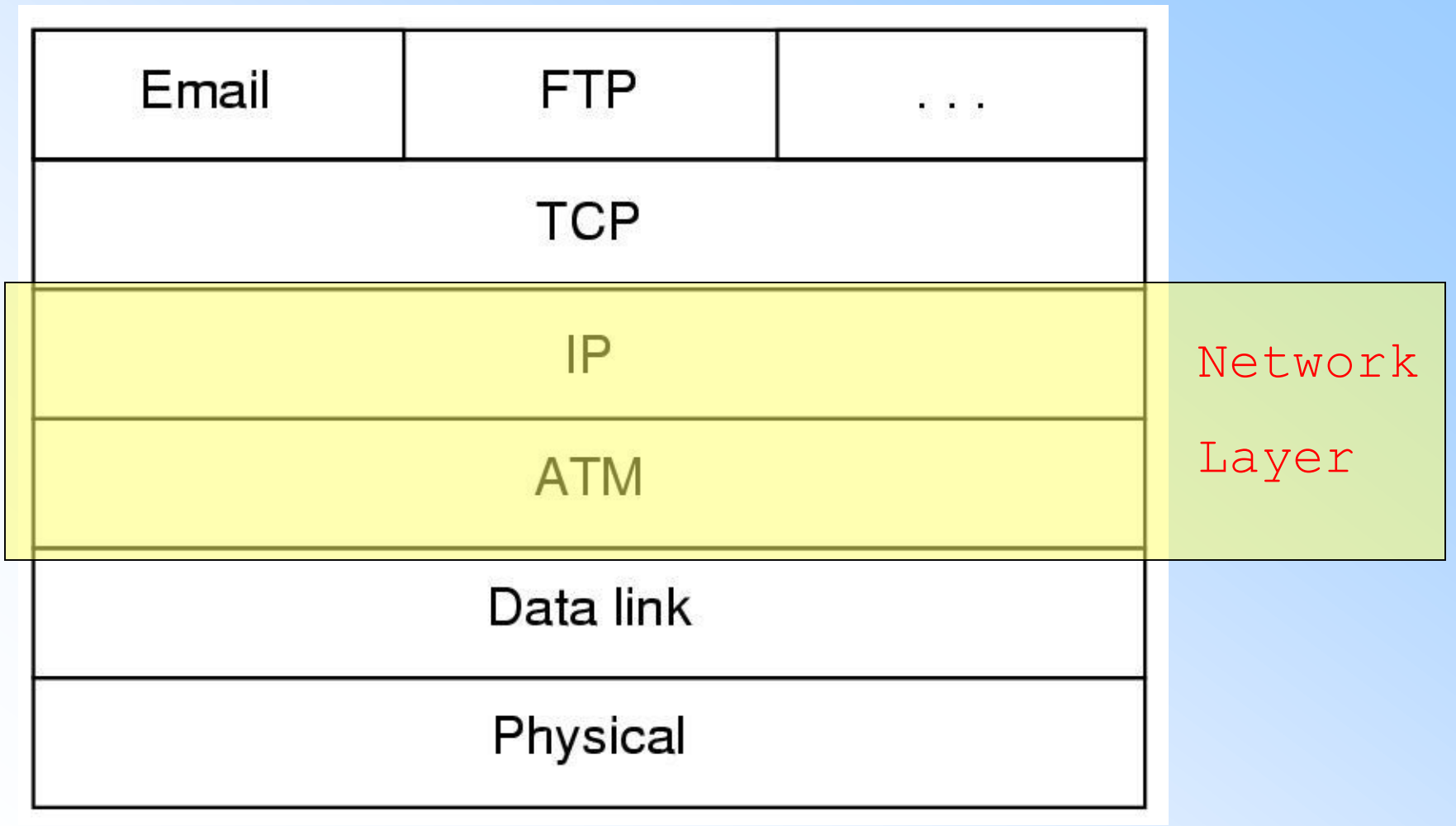
Design issues: services

- Connection-oriented <> Connectionless
  - PTTs: connection-oriented
    - 100 years of experience with the world-wide telephone system
    - Connection time ➔ billing!
  - Internet
    - Subnets are inherently unreliable
  - Real issue: where to put the complexity as some/many applications require reliable transfer (~ connection-oriented service)
    - Network layer
    - Transport layer

Design issues: services

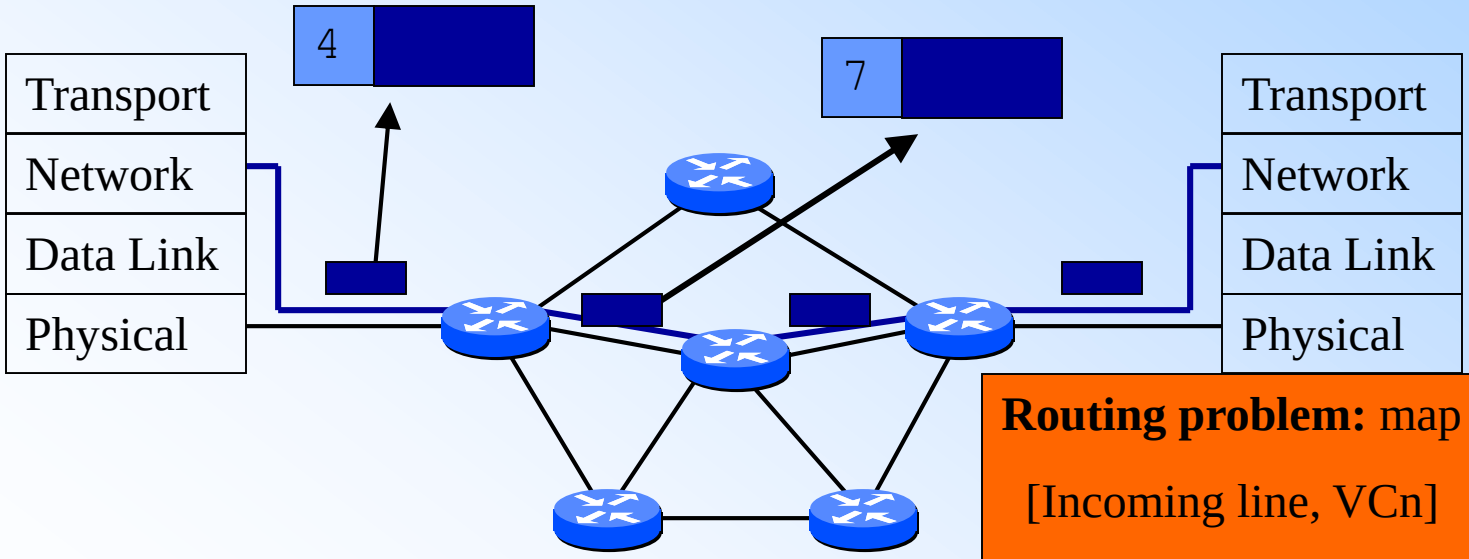
- Connection-oriented <> Connectionless *(cont.)*
  - In favour of connectionless service
    - Computing power is cheap: hosts can handle the complexity
    - Subnet is a large, long lasting investment: keep it simple
    - For some applications speedy delivery (low, constant delay) is important
  - In favour of connection-oriented service
    - Users want a reliable trouble-free service
    - Some services are easier to provide on top of connection-oriented service
  - Examples
    - ATM: connection oriented
    - IP: connection-less
    - IP on top of ATM

Design issues: services



Design: internal organisation

- Virtual circuits
  - Routes chosen at connection time
  - Connection identified by a virtual circuit number (VCn)
  - Primary service of subnet is connection-oriented



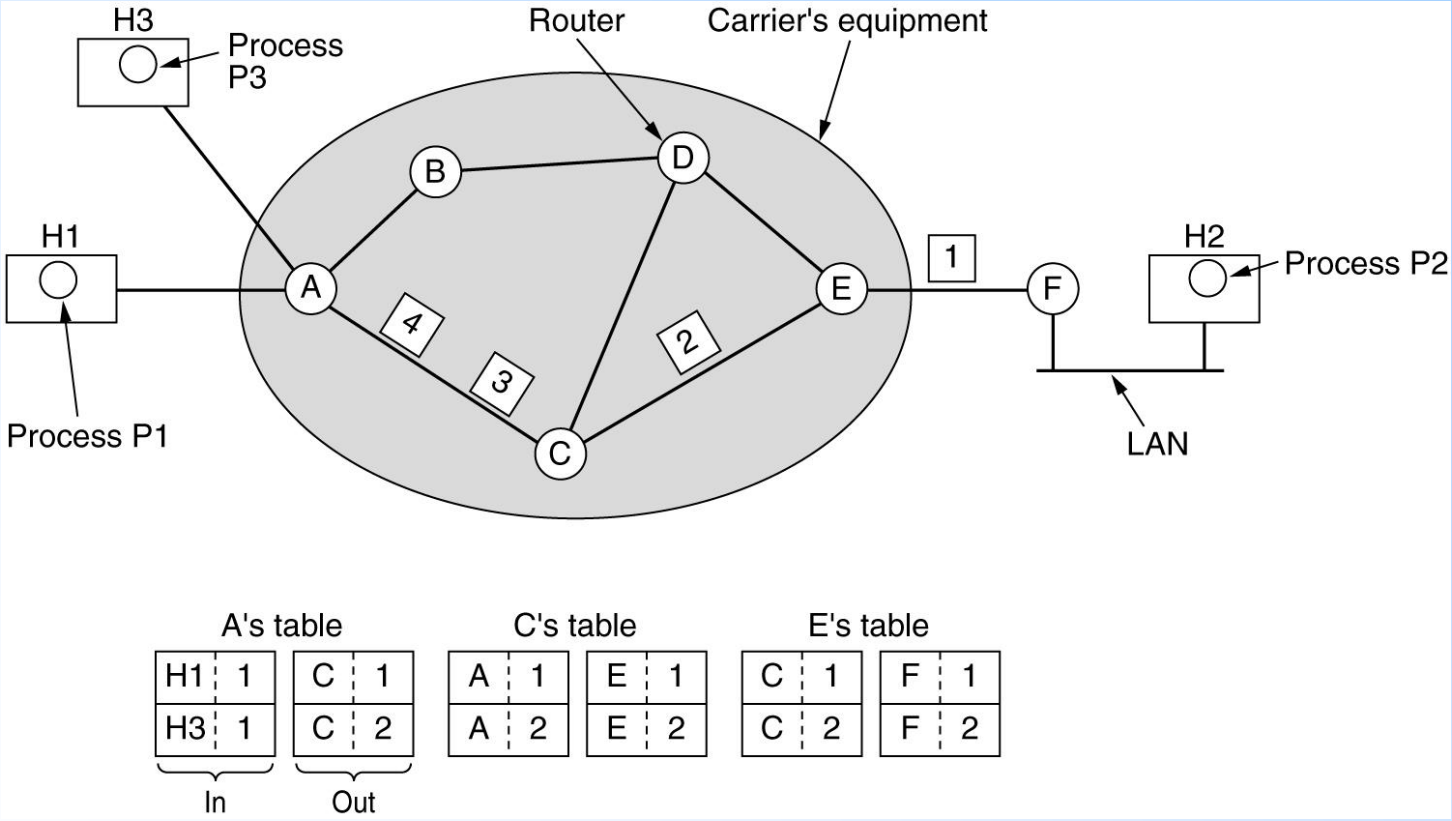
Routing problem: map

[Incoming line, VCn]

➔[outgoing line, VCn]

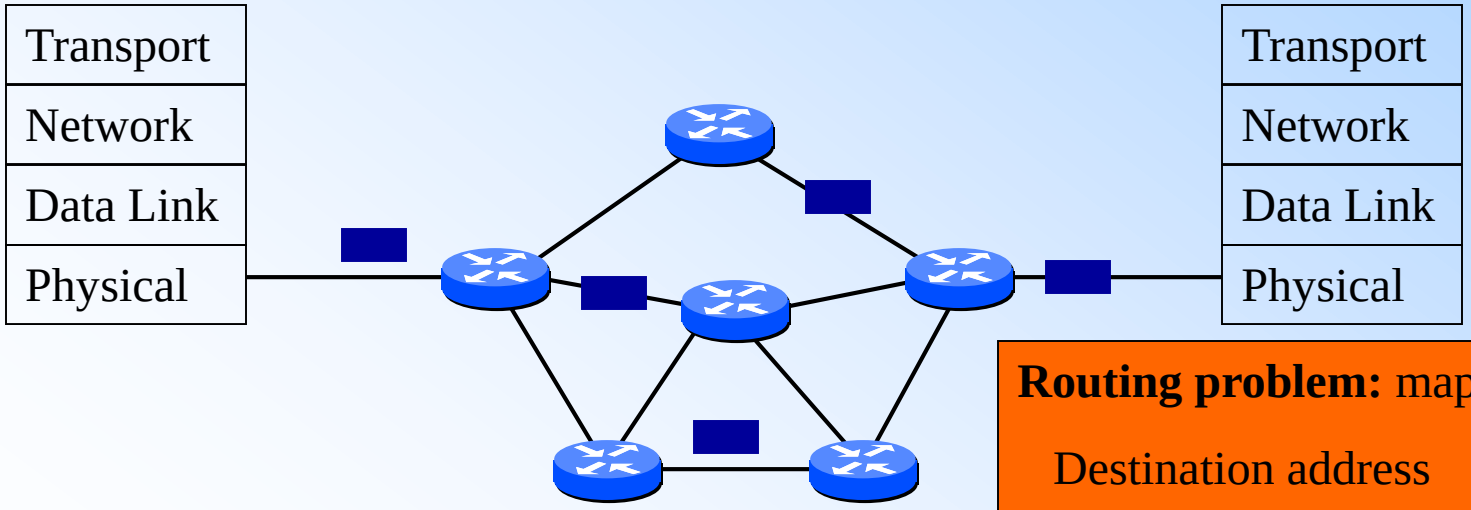
Design: internal organisation

- Virtual circuits



Design: internal organisation

- Datagram subnet
  - Each packet is routed independently
  - Subnet has more work to do
  - More robust, easier to adapt to failures and congestion



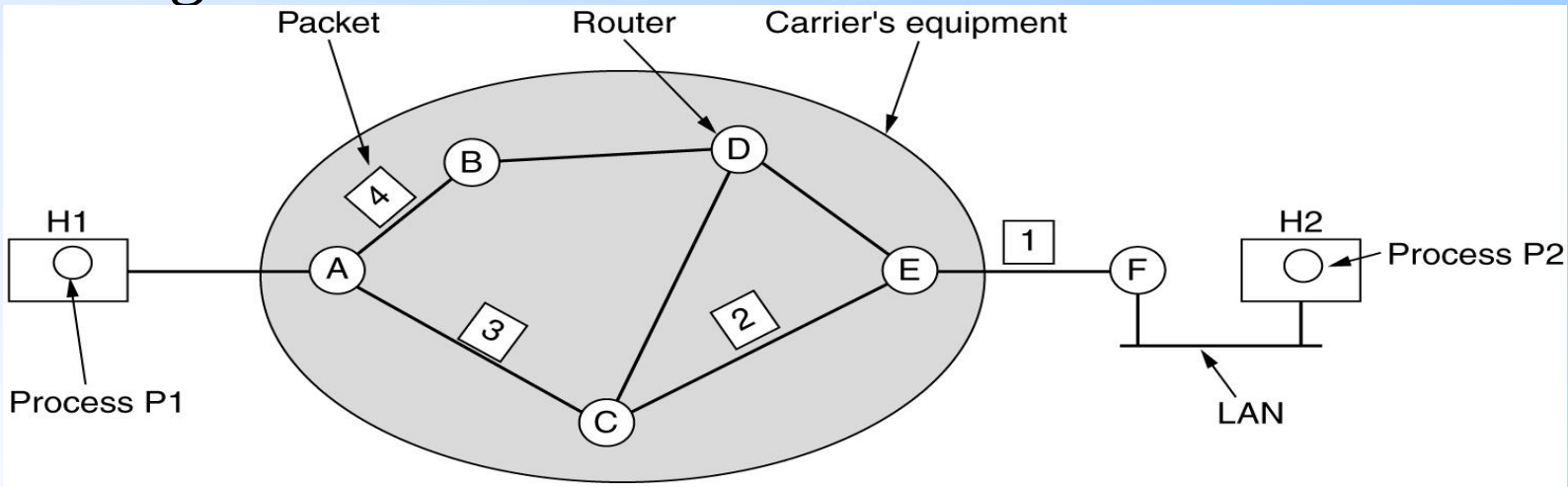
Routing problem: map

Destination address

➔ Outgoing line

Design: internal organisation

- Datagram subnet



A's table initially

Dest.	Line
A	–
B	B
C	C
D	B
E	C
F	C

C's table

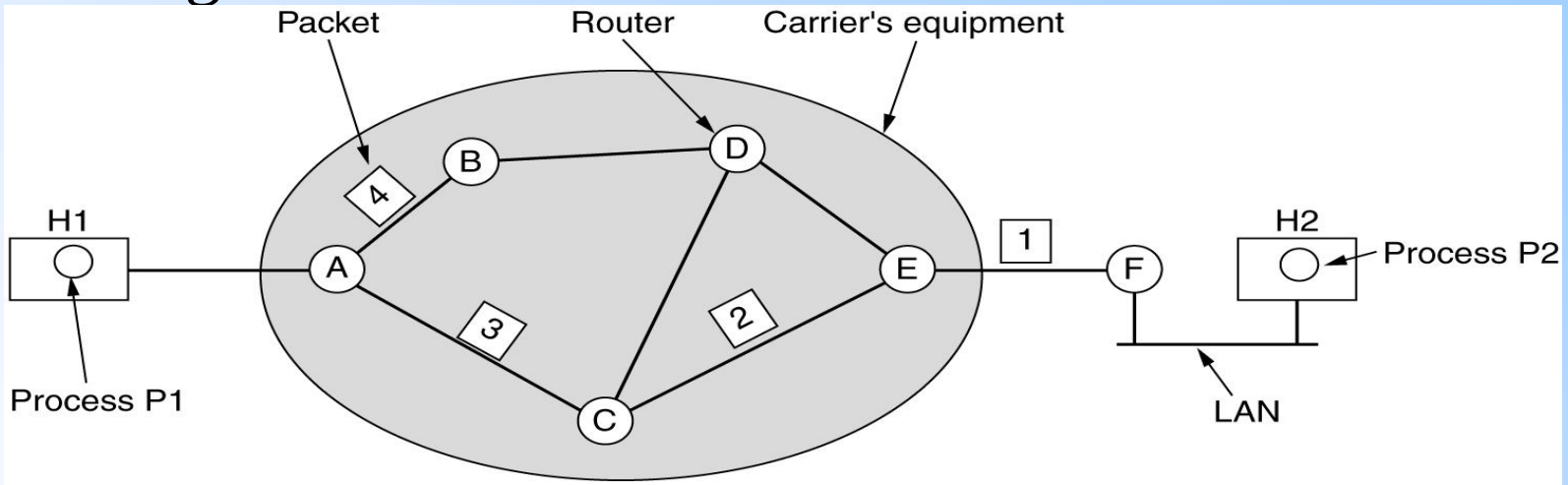
A	A
B	A
C	–
D	D
E	E
F	E

E's table

A	C
B	D
C	C
D	D
E	–
F	F

Design: internal organisation

- Datagram subnet



A's table

Dest.	initially Line	later Line
A	–	–
B	B	B
C	C	C
D	B	B
E	C	B
F	C	B

C's table

Dest.	Line
A	A
B	A
C	–
D	D
E	E
F	E

E's table

Dest.	Line
A	C
B	D
C	C
D	D
E	–
F	F

Change of routing table

Design: internal organisation

Issue	Datagram subnet	VC subnet
Circuit setup	Not needed	required
Addressing	Full addresses (source + destination) in each packet	Short VC number in each packet
Routing	Done for each packet independently	Route chosen at connection setup
Effect of router failures	None, except for packet losses	All VCs passing failed router are terminated
Congestion control	difficult	Easy if enough buffers can be allocated in advance

Network Layer

- Design issues
- Routing
- Congestion
- Internetworking
- Internet Protocols
- Multimedia or Qos

Routing Algorithms

- Properties
- Shortest Path Routing
- Flooding
- Distance Vector Routing
- Link State routing
- Hierarchical routing
- Broadcast routing
- Multicast routing
- Routing for mobile hosts
- Routing in Ad Hoc Networks
- Node Lookup in Peer-to-Peer Networks

Most important algorithms!

Routing algorithms

- Desirable properties
  - Correctness
  - Simplicity
  - Robustness: able to cope with
    - changes in topology, load
    - hardware and software failures
  - Stability
  - Fairness
  - Optimality

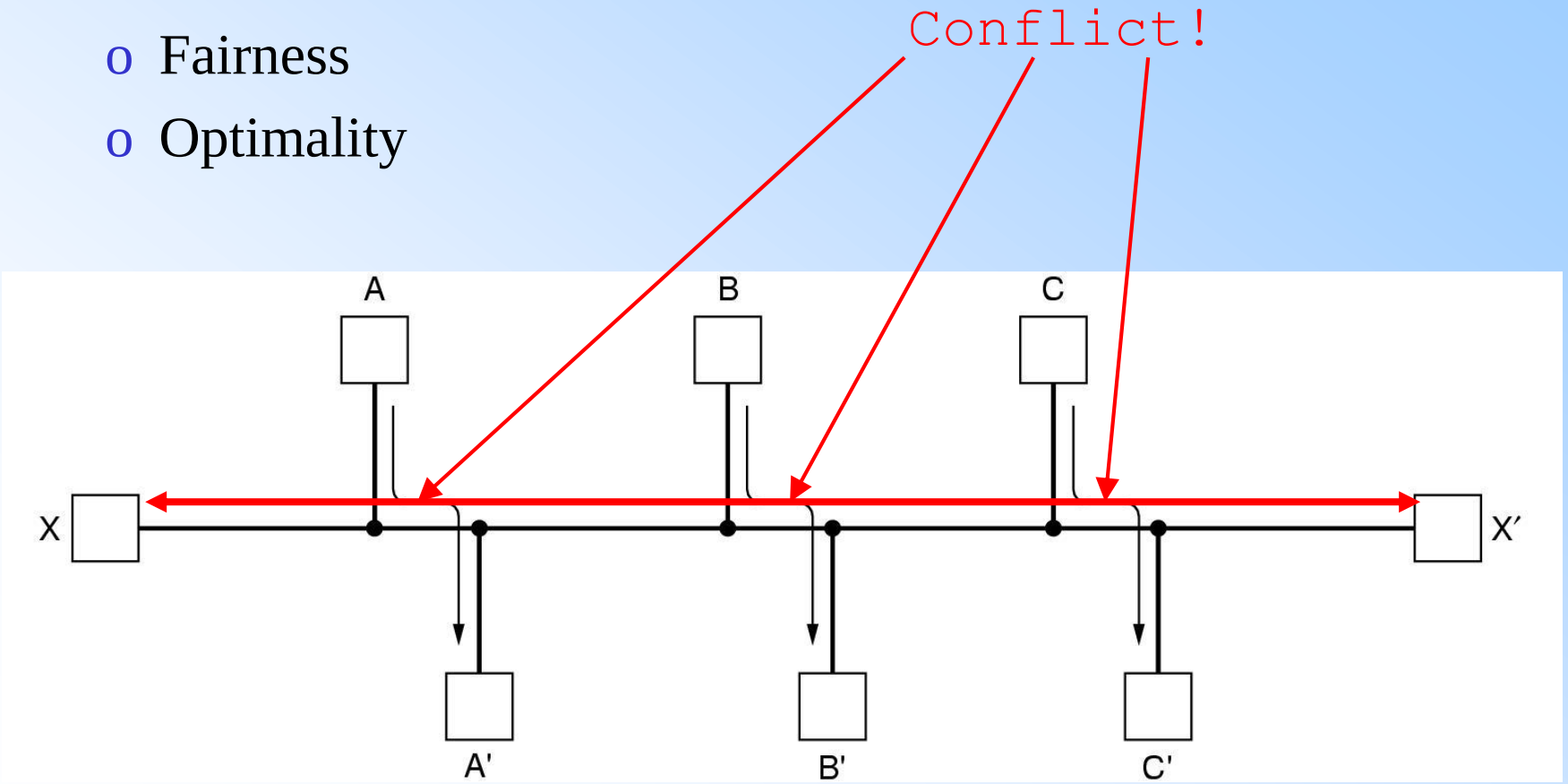
} all algorithms

} hard to achieve

} conflicting

Routing algorithms

- Desirable properties *(cont.)*
  - Fairness
  - Optimality

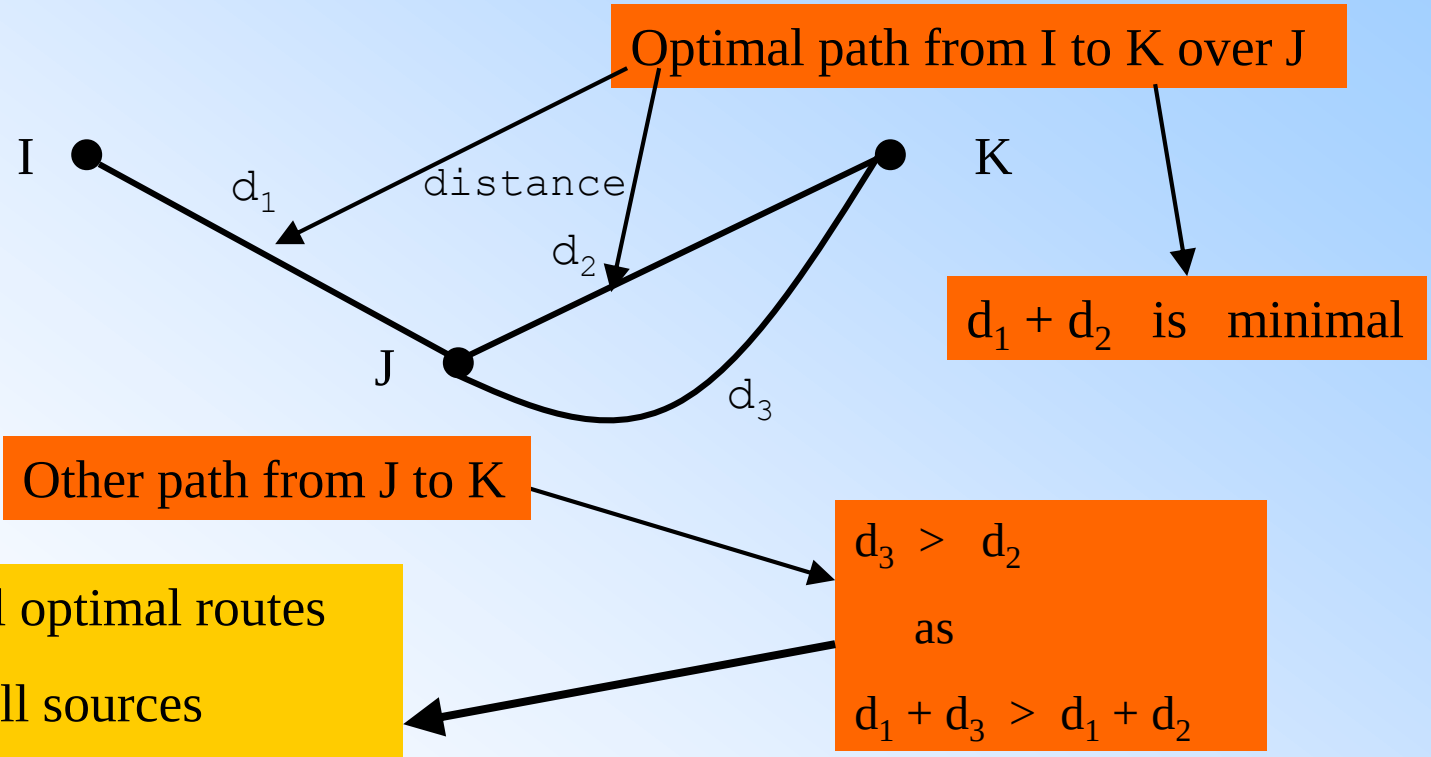


Routing algorithms

- Taxonomy
  - Non adaptive
    - Routing decisions computed in advance, off-line and downloaded
  - Adaptive
    - To changes in
      - Topology
      - Load
    - Get information
      - Locally
      - From adjacent routers
      - From all routers

Routing algorithms

- Optimality principle



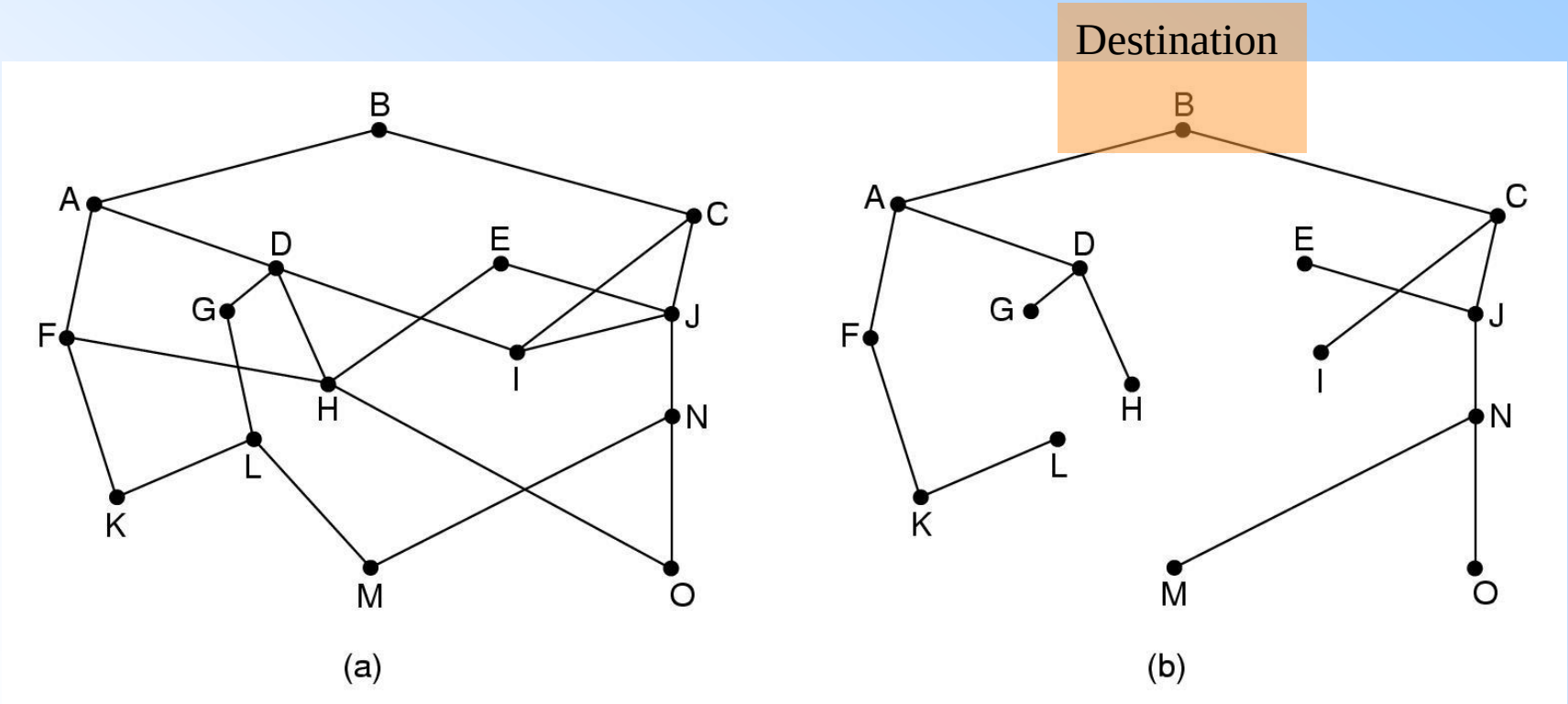
Set of all optimal routes

- from all sources
- to a given destination

is a tree: sink tree

Routing algorithms

- Optimality principle: sink tree

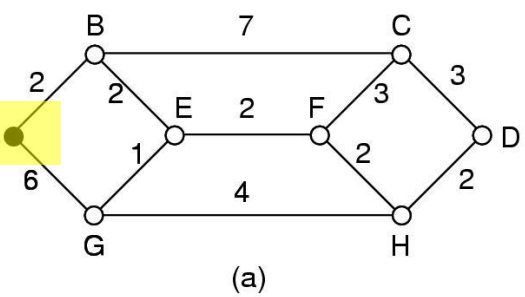


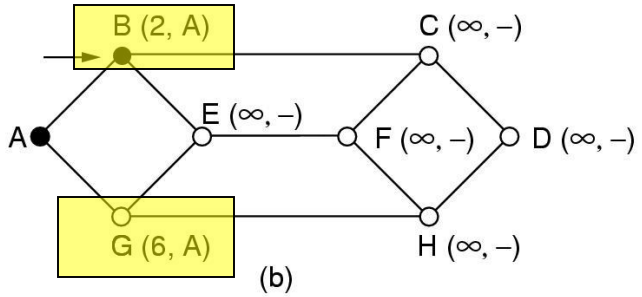
Routing: shortest path

- Algorithm of Dijkstra: shortest path in graph
  - Graph
    - Node = router
    - Arc = communication line
  - Metric
    - Number of hops
    - Geographic distance
    - Mean queueing and transmission delay

Routing: shortest path

Initial node

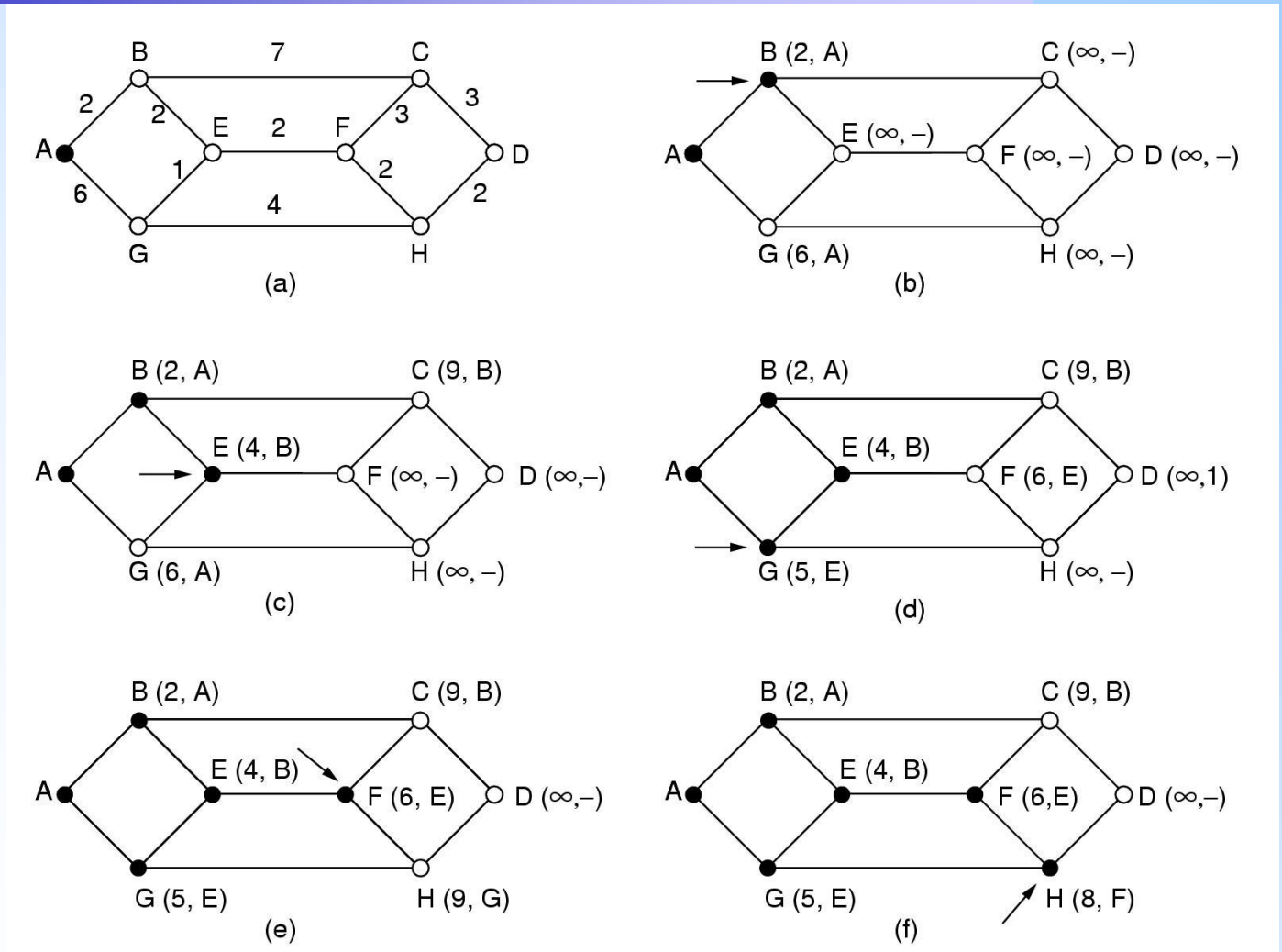




Elements of algorithm:

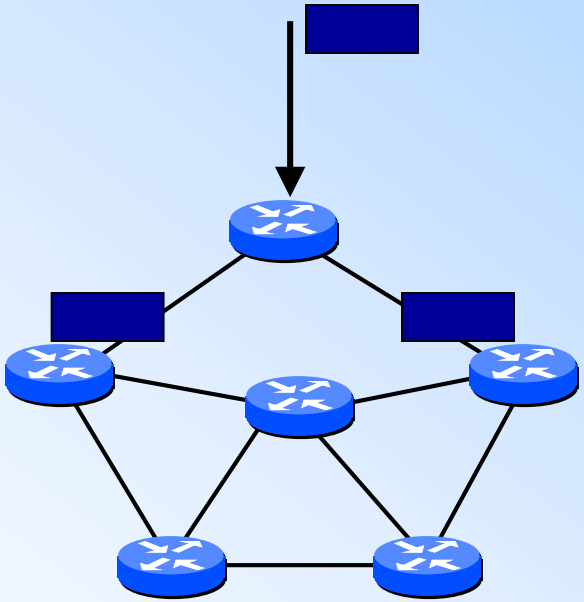
- Mark all nodes as free: O
- Mark initial node as selected: ●
- repeat till destination is selected:
  - Label all free nodes reachable from selected nodes with shortest distance to a selected node
  - Select free node with shortest distance to a selected node and mark it as selected

Routing: shortest path



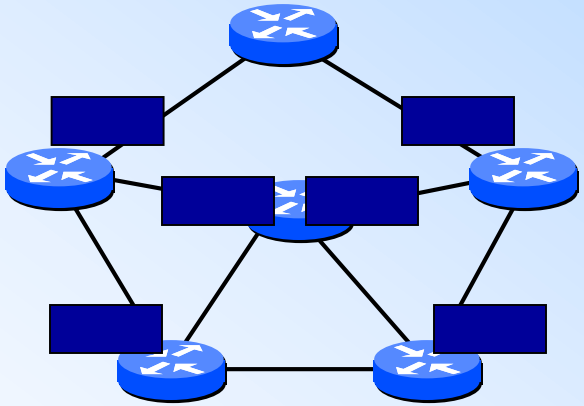
Routing: flooding

- Every packet is sent out on every outgoing line except the one it arrived at



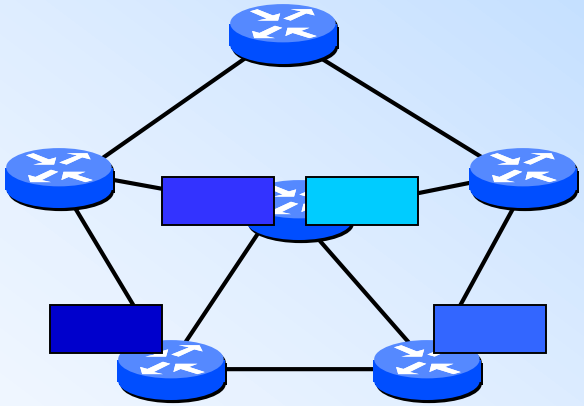
Routing: flooding

- Every packet is sent out on every outgoing line except the one it arrived at



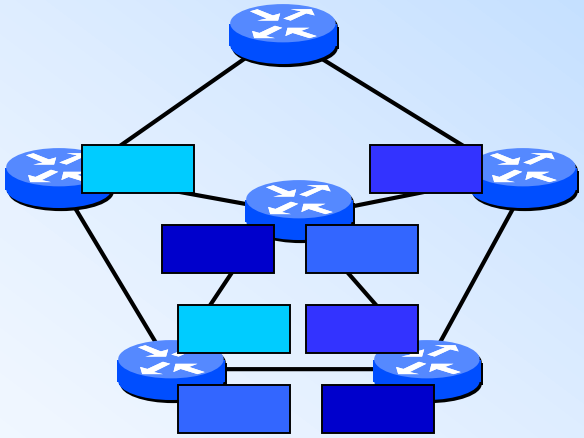
Routing: flooding

- Every packet is sent out on every outgoing line except the one it arrived at



Routing: flooding

- Every packet is sent out on every outgoing line except the one it arrived at



Routing: flooding

- Every packet is sent out on every outgoing line except the one it arrived at
- Duplicates!! How to limit?
  - Hop counter
    - Decrement in each router
    - Discard packet if counter is 0
    - Initialisation?
  - Sequence number in packet
    - Avoid sending the same packet a second time
    - Keep in each router per source a list of packets already seen
- *Useful?*

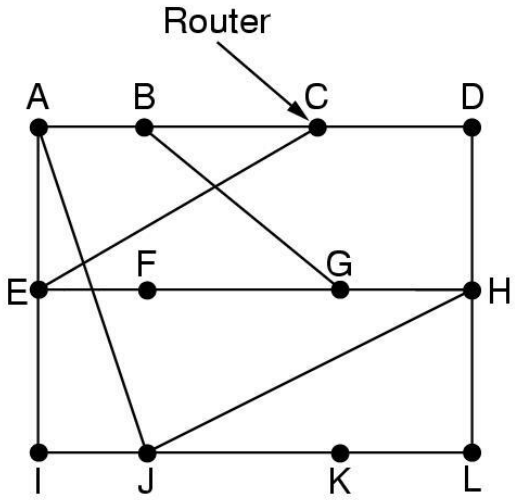
Routing: flooding

- Every packet is sent out on every outgoing line except the one it arrived at
- Sometimes useful
  - Robust algorithm: e.g. military applications
  - Broadcast
  - Comparison purposes: always shortest path
- Selective flooding
  - Use only those lines that are going approximately in right direction
  - Still working?

Routing: distance vector

- Adaptive algorithm
  - Exchange of info only with neighbours
- Data to be available in each router
  - Routing table: per destination
    - Distance
    - Outgoing line
  - Distance to all neighbours

Routing: distance vector



(a)

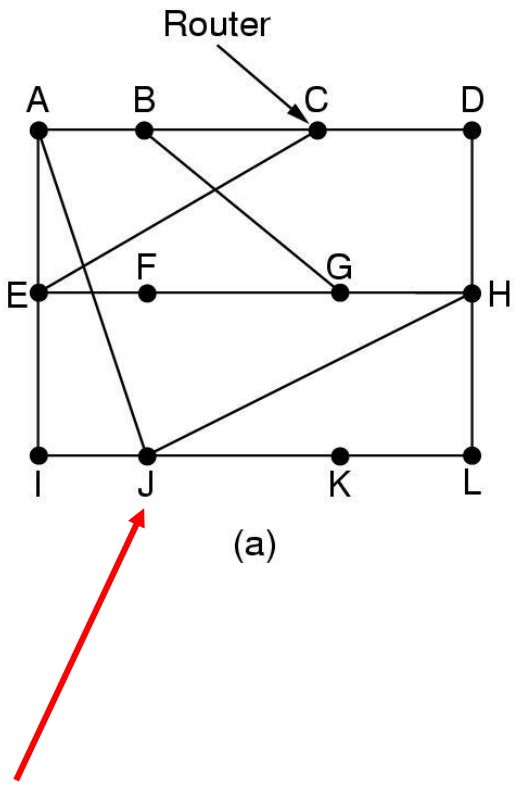
Routing table for A

To	cost	via
A	0	-
B	12	B
C	25	B
D	40	B
E	14	E
F	23	E
G	18	B
H	17	J
I	21	E
J	9	J
K	24	J
L	29	J

Routing: distance vector

- Algorithm
  - At each step within a router:
    - Get routing tables from neighbours
    - Compute distance to neighbours
    - Compute new routing table

Routing: distance vector



(a)

Routing tables from

To	A	I	H	K	New estimated delay from J	Line
G	18	31	6	31	18	H

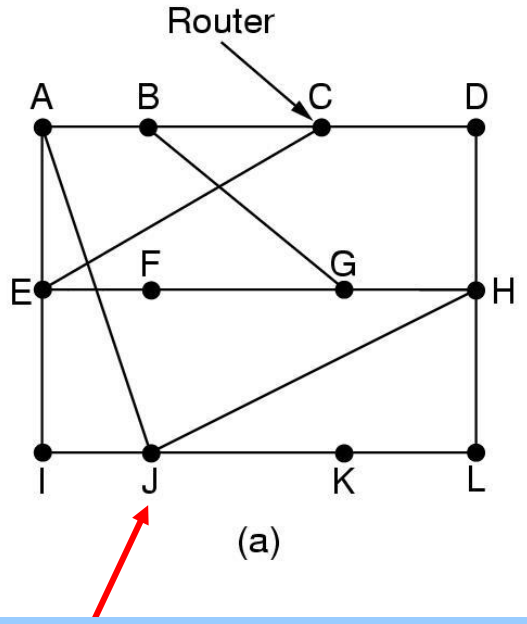
JA delay is 8 | JI delay is 10 | JH delay is 12 | JK delay is 6

Vectors received from J's four neighbors

New routing table for J

(b)

Routing: distance vector



To	A	I	H	K	New estimated delay from J	Line
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	–
K	24	22	22	0	6	K
L	29	33	9	9	15	K

JA delay is 8 | JI delay is 10 | JH delay is 12 | JK delay is 6

Vectors received from J's four neighbors

New routing table for J

(b)

Better?

- Keep 4 tables (one for each neighbour)
- Use shortest path

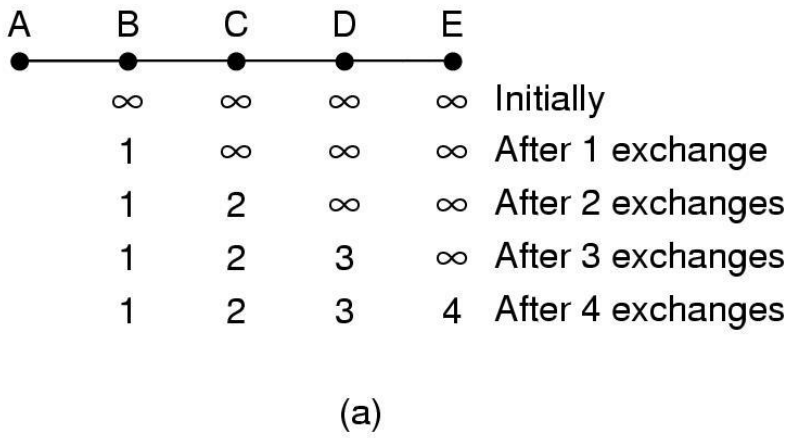
Routing: distance vector

- Distributed algorithm
  - Triggers:
    - Change in delay to neighbour
    - Receive new table from neighbour
  - Update local tables
  - If changed: forward routing tables to neighbours

How fast are changes propagated?

- Good news?
- Bad news?

Routing: distance vector



(a)

Good news:

- A comes up again

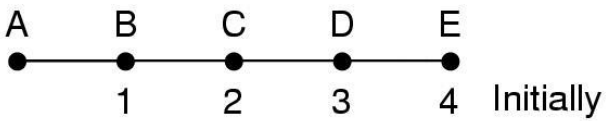
Only distances to A

Faster not possible!!!

Routing: distance vector

Bad news:

- A goes down



B receives:

- Distance ∞ from A
- Distance 2 from C

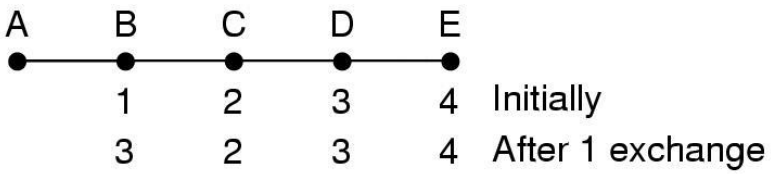
New distance from B to A: 3 via C

(b)

Routing: distance vector

Bad news:

- A goes down



C still

- believes its distance to A is 2
- routes via B

B routes its packets for A via C

(b)

LOOP!!!

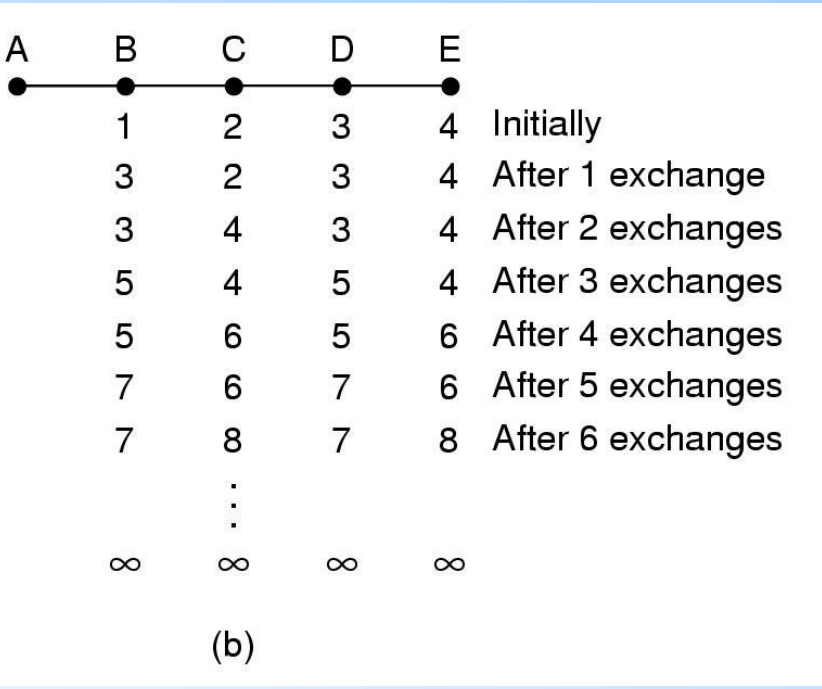
Count to infinity problem

Bad news:

- A goes down

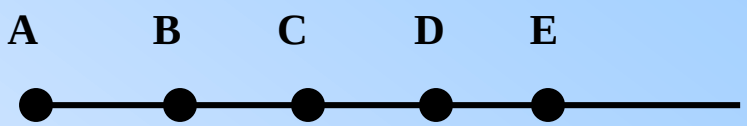
Loops!!

Slow!!



Routing: distance vector

- Solution to count to infinity:
  - Define infinity
  - Split horizon
  - Split horizon and poison distribution



1	2	3	4	initially
∞	2	3	4	1 iteration
∞	∞	3	4	2 iterations
∞	∞	∞	4	3 iterations

Routing: link state

Overview of algorithm:

- Each router must
  - Discover its **neighbours** and **learn** their network addresses
  - Measure the delay or **cost** to each of its neighbours
  - Construct a **packet** with these distances
  - Send this packet to **all** other routers
  - Compute the **shortest path** to every other router

Routing: link state

Algorithm:

- Learning about neighbours:
  - Upon boot of router
    - Send HELLO packet on each point-to-point line
    - Routers are supposed to send reply with a globally unique name

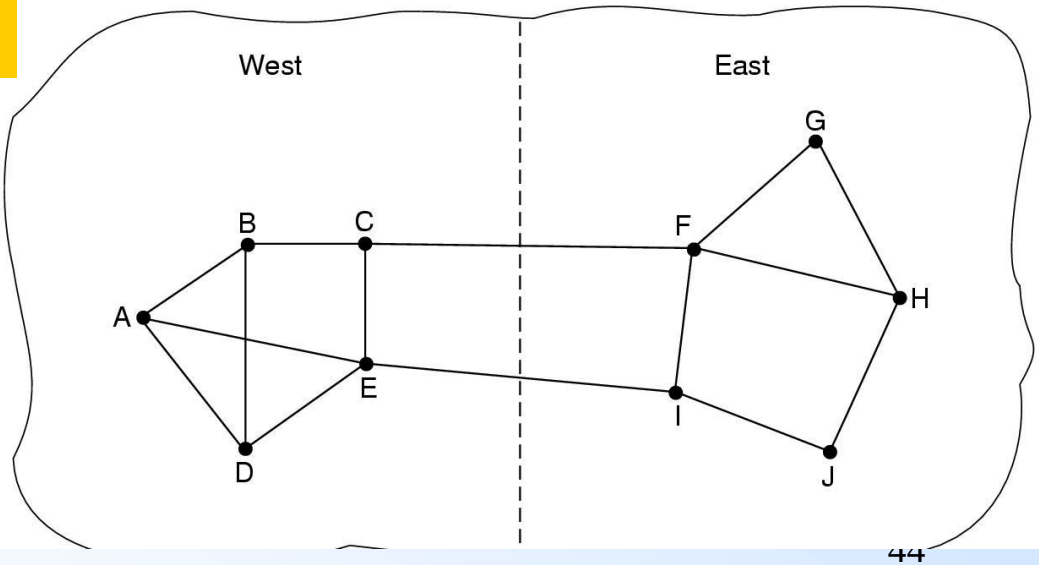
Routing: link state

Algorithm:

- Measuring line cost
  - Measure round-trip delay of HELLO Packet and its reply
  - Take load into account? Arguments both ways:

Yes! preference for unloaded line

No! oscillations are possible

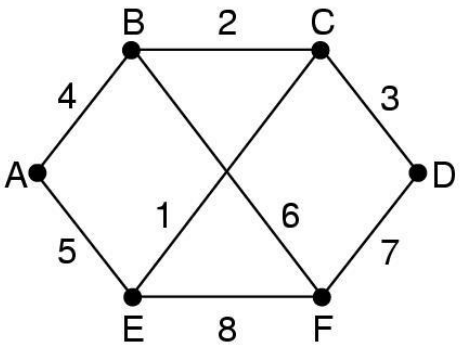


KU Leuven
DistriNet
Research Group

Routing: link state

Algorithm:

- Building link state packets
  - Packet containing:
    - Identity of sender
    - Sequence number + age
    - For each neighbour: name + distance
  - When to build?
    - periodically
    - when significant events occur



(a)

Link State Packets

A	B	C	D	E	F
Seq.	Seq.	Seq.	Seq.	Seq.	Seq.
Age	Age	Age	Age	Age	Age
B 4	A 4	B 2	C 3	A 5	B 6
E 5	C 2	D 3	F 7	C 1	D 7
	F 6	E 1		F 8	E 8

(b)

Routing: link state

Algorithm:

- Distributing link state packets
  - Trickiest part of algorithm
    - Arrival time for packets different
    - How to keep consistent routing tables
  - Basic algorithm
    - Flooding +
    - Sequence number (in each packet) to limit duplicates
  - Manageable problems
    - Wrap around of sequence numbers: 32 bits + 1 packet/sec ➔ 137 years
    - Wrong sequence number used:
      - lost in case of crash
      - Corruption

Age in each packet:

- Decremented during flooding, while used in router
- Age 0 ➔ info discarded

Routing: link state

Algorithm:

- Distributing link state packets
  - Basic algorithm
  - Manageable problems
  - Refinements
    - Link state packets are not forwarded immediately
    - During holding time:
      - duplicates are discarded
      - Old packets are thrown out

Routing: link state

Algorithm:

- Computing new routes:
  - With a full set of link state packets, a router can:
    - Construct the entire subnet graph
    - Run Dijkstra's algorithm to compute the shortest path to each destination
  - Problems for large subnets
    - Memory to store data
    - Compute time

Routing: link state

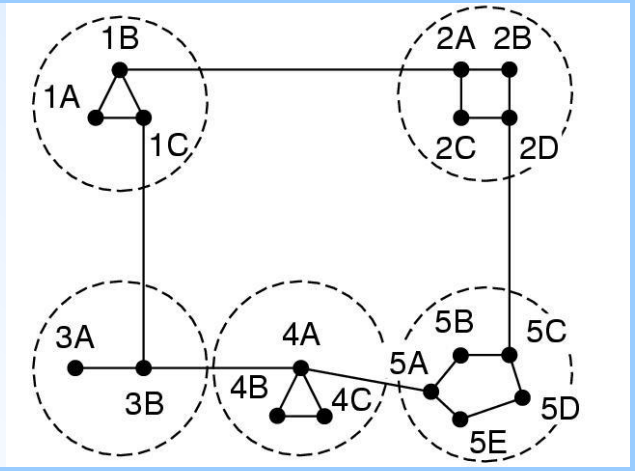
- Usage:
  - IS-IS protocol
    - Designed for DECnet, adopted by ISO
    - In use also in Internet
    - Supports multiple network layer protocols
  - OSPF protocol used in Internet
  - Common features:
    - Self-stabilizing method of flooding link state updates
    - Concept of a designated router on a LAN
    - Method of computing and supporting path splitting and multiple metrics

Routing Algorithms

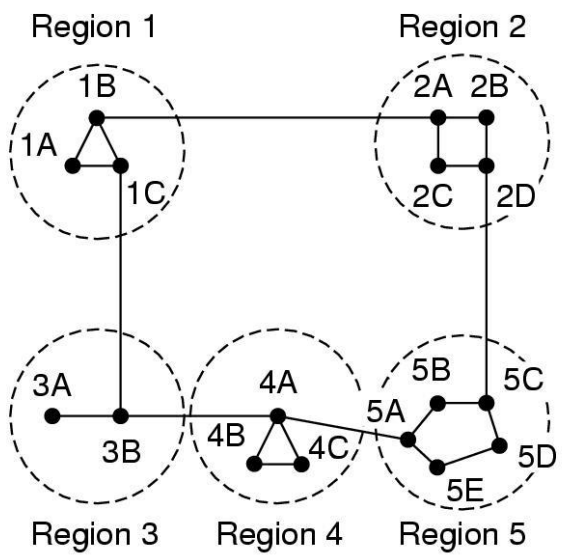
- Properties
- Shortest Path Routing
- Flooding
- Distance Vector Routing
- Link State routing
- Hierarchical routing
- Broadcast routing
- Multicast routing
- Routing for mobile hosts
- Routing in Ad Hoc Networks
- Node Lookup in Peer-to-Peer Networks

Hierarchical routing

- When network size increases…
  - Larger stables
  - More CPU time needed to compute …
  - More bandwidth needed



Hierarchical routing



(a)

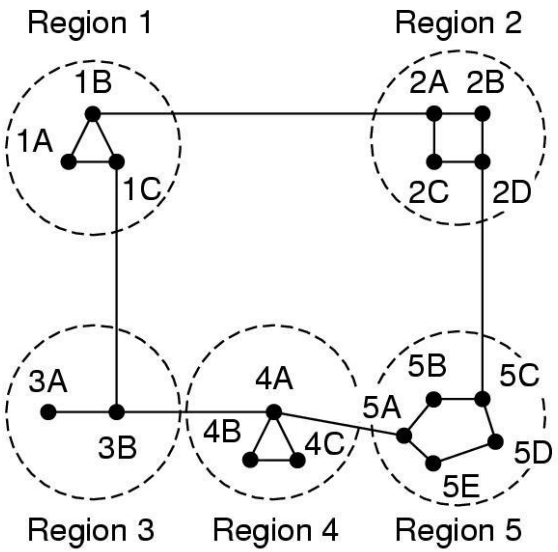
Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

- Solution?

Hierarchical routing



Full table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Hierarchical table for 1A

Dest.	Line	Hops
1A	–	–
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

- Routers grouped in regions
- Each routers knows how to reach:
  - Other routers in its own group
  - Other regions

\+ Smaller tables

– Longer paths

Hierarchical routing

720 Routers
720 table entries (No hierarchy)

If subnet is partitioned into 24 regions of 30 routers each:
Each router will have
30 entries for routers plus 23 entries for regions=53 entries

If a three level hierarchy is chosen, with eight clusters, each containing 9 regions of 10 routers each.
Each router will have
10 entries for routers plus 8 entries for regions plus 7 entries for clusters =25 entries